

Welcome Email

This task intends to provide you with experience in Node.js, Express and Web APIs. You are given the requirements of sending a welcome email to new subscribers who join the DEV@Deakin web platform that you are creating.

Complete the following:

1. Complete the Task Requirements.
2. You will find “Demo Videos” of Week 2 on the SIT313 CloudDeakin site particularly useful as a reference for this task.
3. Create a new [GitLab](#) repository for this task. Ensure all files/resources created are committed to this repository.
4. Create a 2 – 4 minute video where you provide a walkthrough of your code and solution, demonstrating your understanding of the concepts. Upload your video to [Panopto](#).
5. Create a short report that includes:
 - a. Screenshots of your DEV@Deakin platform.
 - b. Screenshots of your terminal, showing a successful Status 200 message from your chosen Email API (hint: may need to use `console.log()`).
 - c. A link to your task repository on GitLab.
 - d. A link to your video.
6. **Upload** to OnTrack and **engage with feedback** to get this complete!

Task Requirements

1. GitLab repository

- Create a [GitLab](#) account (using GitLab at Deakin). You should already have an account based on your student username.

- Create a New Project called Task P2. **Set the Visibility Level to *Internal*.** Failure to do this means you cannot have your submission marked as Complete.
- In your repository you **must** create a `.gitignore` file, and add inside it `node_modules/` to prevent this folder from being tracked by Git.

Strictly do not include your `node_modules` folder in your GitLab repository. Failure to do so means you will receive a Resubmit for this task.

All code and resources (e.g. images) that you develop for this task must be committed to your Task P2 repository, including proper commit comments.

2. Sign-Up Feature

Expand on your website you started in task P1 by adding a sign-up feature, similar to the below example. For now, use only CSS and HTML, although you can explore the **Optional Technologies** section for alternatives.



SIGN UP FOR OUR DAILY INSIDER

Users can enter in their email and click the Subscribe button. Your Node.js backend should detect any new sign-ups, and add their email to the mailing list.

3. ExpressJS Backend

Create an ExpressJS backend that your frontend connects to. You have multiple options for sending HTTP requests between your frontend and backend:

- Use the *action* and *method* properties of a `<form>`
- Use JavaScript *fetch()* library

Each time you modify your `.js` file, you must restart the ExpressJS service so the changes take effect. Visit the **Optional Technologies** section for a useful package to automate this process for you.

Your backend should:

- Handle user sign-ups to your email from the frontend.
- Utilise an Email API to send emails asynchronously to signed-up users.
- Send proper [HTTP response status codes](#).

You can use an Email API such as [SendGrid](#), [MailGun](#), or others. There are lots of resources available to assist you with this task, such as on the SIT313 CloudDeakin site, or perhaps this YouTube [tutorial about SendGrid](#).

Important: Never store API keys or sensitive data in your code directly, instead add them to a `.env` file. Be sure to force Git to not track your `.env` file by adding it to your `.gitignore`.

Optional Technologies

We strongly encourage you to explore additional technologies that are often standard in industry, that may assist you in completing this task:

- [Nodemon](#) (Node Monitor) package – Automatically restarts your ExpressJS server each time code changes are detected. Highly recommended.
- [Tailwind CSS](#): A CSS framework where you combine your CSS code with the HTML code. Allows you to design your pages faster.
- CSS Flexbox – A layout model for arranging your HTML elements in a much more flexible, dynamic way. Here is a [great guide](#).
- CSS Grid – A layout model for arranging your HTML elements in a grid-layout. Here is a [great guide](#).
- [Bootstrap](#) – A well-established and known CSS framework, with many pre-made templates for forms, buttons, navigation, and other user interface components.

Know of some awesome technologies? Let us know in the SIT313 Teams channel!

Optional VS Code Extensions

VS Code has a gigantic library of extensions that can improve and speedup the development process. Here are some that may assist you:

- **Live Server** by Ritwick Dey – Automatically refreshes your webpage each time you make a change in your HTML/CSS code.
- **Tailwind CSS IntelliSense** by Tailwind Labs – If using Tailwind CSS, this is a must-have extension to enable auto-complete when typing.
- **Prettier** by prettier.io – Automatically formats your codes' style and structure, to enforce a consistent style throughout.

Available Resources

Feeling a bit lost?

No problem at all! Your tutors are experts and would love to assist. Ensure to attend the (you can attend as many seminars as you like) and speak with a tutor, or write a message on the SIT313 Teams channel.

We have many resources to assist in your learning journey – if you are ever unsure then please reach out to us, we are always happy to help.

- **The SIT313 Teams channel** – an active learning resource with weekly announcements from the Unit Chair, and regular discussions/assistance from the SIT102 teaching team and students.
- **Seminars** – you can attend both on-campus and online sessions, and join as many as you like each week. Your tutors will provide demonstrations and assistance to help guide your learning.
- **SIT313 CloudDeakin** – contains a large collection of text and video resources.
- **Announcements** - Keep an eye on your email and any announcements made in the SIT313 Teams channel.

Caution: Acceptable use of Artificial Intelligence

It is acceptable to use AI as a learning tool. However, you can **never** ask AI to write code to complete some/parts of this task for you: this is cheating because it does not demonstrate that you understand anything. Detection of inappropriate use of AI has a very real consequences, such as failing the task, or even failing the entire unit.